So far, you have built a web application with Spring Boot (Catalog Service), containerized it with Cloud Native Buildpacks, and run it with Docker. To complete the pilot project for Polar Bookshop, you must perform one last step: deploy the application to a cloud environment. To do that, you will use Kubernetes, which has become the de facto standard for container orchestration. I will provide more details about Kubernetes in later chapters, but I want you to have a first taste of how it works and how you can use it to deploy a web application.

Kubernetes (often shortened as *K8s*) is an open source system for automating the deployment, scaling, and management of containerized applications (https://kubernetes.io). When you are working with containers in Docker, your deployment target is a machine. In the example from the previous section, it was your computer. In other scenarios it might be a virtual machine (VM). In any case, it's about deploying containers to a specific machine. However, when it comes to deploying containers without downtime, scaling them by leveraging the cloud's elasticity, or connecting them across different hosts, you'll need something more than a container engine. Instead of deploying to a specific machine, you are deploying to a cluster of machines, and Kubernetes, among other things, manages a cluster of machines for you. I covered this distinction in the previous chapter in the context of topologies. Figure 2.9 will remind you of the different deployment targets in a container topology and an orchestration topology.